punt-kit Is the Kit Manager for the Punt Labs Org

One tool for what the org knows and what the org does — its standards and compliance checks, its release workflow and playbooks, and the kit of tools a repo runs on — for the operator and the AI agents working roughly thirty repositories

`/punt:auto` turns recurring prompts into reviewed programs: playbooks that mix deterministic shell with LLM judgment, and — next — prompt state machines that drive the agent instead of the operator retyping the loop

Press Release

PUNT LABS — August 2026 — punt-kit is the kit manager for the Punt Labs org. It manages four things: the org's **standards** (author them, activate them in a repo, audit against them), its **workflows** (the 11-phase release engine and the playbooks), its **compliance** checks (convention audit, PII scan), and — as the direction it is now building toward rather than a shipped command — the **kit of tools** each repo runs on. The customer is Punt Labs itself: one operator and the AI agents that do the work across roughly thirty repositories. External adoption is not a goal and is explicitly deferred (see FAQ 1, FAQ 9).

The centerpiece is `/punt:auto`, and the direction it is headed is a **self-prompting engine**: recurring prompts the operator types by hand become playbooks — reviewed, versioned programs that generate the prompts and drive the agent. The executor and six playbooks ship today, the release engine among them. Turning that into a general engine for loops is the next investment, committed but not yet built (see FAQ 4).

Problem

Punt Labs is one operator and a roster of AI agents working roughly thirty repositories. Two costs dominate, and both scale with the number of repos and sessions rather than with the amount of code.

The first is context. Every agent session starts cold, with no memory of what came before [1], and the org's conventions — how a release runs, what a README must contain, when a PR may merge — live in prose that each session has to find. A study of 253 CLAUDE.md files found these manifests are optimized for code execution rather than organizational convention: only 8.7% include security guidance [2]. Static context files also have no feedback loop, so nothing notices when a repo has drifted from the document that describes it [3]. Catching that drift in review is the expensive path — a static-analysis vendor puts rote standards checking at roughly four hours per engineer per week [4], which is a figure to treat as directional, since the vendor is quantifying the pain its own product removes.

The second is the operator typing the same prompt again. Watching a pull request to merge, sweeping the backlog, running a release — these are loops with named states and guarded transitions, and today the operator re-authors each one as a fresh cron prompt, once per PR. A hand-typed loop is unreviewable, unversioned, and evaporates when the session compacts. It is the same toil the standards corpus removed for conventions, still untouched for procedure (see FAQ 4).

Solution

An org run by agents has to manage two things, and they turn out to be the same problem. What the org **knows** — its conventions, its rules, what "good" looks like — has to reach a session that starts with no memory of it. What the org **does** — its releases, its review loops, its backlog sweeps — has to reach that session too, and today it arrives as a prompt somebody retypes. punt-kit manages both: the standards corpus is the knowing, the playbooks are the doing, and the four pillars below are how each of them gets into an agent's hands.

Standards. The corpus is 29 normative prose documents in `standards/`, plus per-language coding rules in `lang-rules/` that agents auto-load when they touch a matching file. They are prose on purpose: the consumer is a language model and a human reviewer, and neither needs the rules ground into assertions first. What is machine-checkable is checked separately and deterministically — `punt audit` is hand-written Python that verifies the org's conventions in a repo (CI workflows, tool config, permissions, Makefile targets, README SHA pins, plugin version sync, branch-protection rulesets) and reports each finding with its rule. `punt init` scaffolds a new repo that already satisfies them.

Tools — the direction, stated as such. Punt Labs ships around a dozen of its own tools, and a repo should be able to say which ones it uses. `standards/tool-enable-disable.md` defines how: each tool owns the vendored zone and the `enabled` marker of its `.punt-labs/<tool>/` subtree, and the host `CLAUDE.md` carries exactly one `@`-import line per enabled tool, so that tool's own guide composes into every session. `enable` and `disable` are the verbs, and `disable` is non-destructive.

What is true today: the architecture is specified and adopted — biff, ethos, and vox each implement `enable/disable` in their own CLI, and the markers are committed in nine repos for biff and three for ethos. *What is not:* punt has no manager command. At v0.11.2 the CLI is `init`, `audit`, `pii`, `release`, `auto`, `doctor`, `status`, and `version` — there is no `punt install` and no `punt enable`. That surface is the invested next step, not a shipped one (see FAQ 2, Feature 8).

Workflows. `punt release` is an 11-phase release engine — preflight, version bump, build, release PR, tag, CI wait, GitHub release, PyPI verification, post-release, cross-repo propagation, and a final verify pass. It runs on every release, and in the operator's judgment it is the most-used feature in the org — judgment rather than measurement, because nothing logs usage yet (FAQ 18).

Compliance. `punt audit` and `punt pii` are the deterministic checks: convention conformance and a scan for emails, home directory paths, and hostnames, with a `-staged` mode for pre-commit.

The engine underneath. `/punt:auto` runs playbooks: YAML programs whose steps are either deterministic shell or scoped LLM judgment, with postconditions and a per-step failure strategy. The executor is an LLM following `skills/auto/SKILL.md` — not a daemon — which is what lets a judgment step sit next to a shell step in one program. The release engine is the shipped proof: eleven deterministic phases with LLM diagnosis on failure. The drafted next step turns the DSL into a prompt state machine — named states, guarded transitions, poll ticks, and persisted run-state with crash/resume — so a loop is a reviewed file rather than a retyped prompt (see FAQ 4).

Where this is going. *Shipped today: the CLI and the plugin. The rest of this paragraph is roadmap, not capability.* The destination is the engine-and-clients model in `standards/architecture.md` — one engine behind thin client surfaces — and two of those clients are committed and unbuilt. An **MCP server** will expose audit, status, and playbook launch to any agent runtime, not only Claude Code. A **REST surface** will serve CI jobs and dashboards that need findings programmatically. The point of building both is that the checks and the playbooks do not fork per caller: one engine answers a human at a terminal, an agent over MCP, and a CI job over HTTP identically (see FAQ 7, Feature 14, Feature 15).

Customer Quote

> *"I run the releases, and every one of them goes through `/punt:auto release` — eleven phases, and when a phase fails the executor reads the error and tries a fix instead of handing me a stack trace. That is the part I no longer think about. The part I still think about is the watching: I hand-type the same PR poll prompt every time a PR opens, and I typed it twice in one session last week. Those prompts are a state machine I re-derive from memory each time. I want to run the file instead."*
>
> — **Claude Agento**, COO/VP Engineering agent, Punt Labs

Getting Started

Install with `install.sh` (or `uv tool install punt-kit` if uv is already present). In a new repo, `punt init` scaffolds CI workflows, tool config, permissions, and a `CLAUDE.md` that references the standards. `punt audit` reports where an existing repo diverges. `/punt:auto list` shows the playbooks available, and `/punt:auto release` runs a release end to end. Nothing phones home; every command runs locally against the checkout in front of it.

Spokesperson Quote

> "Our standards stay prose, and that is a decision, not a gap. The consumer of a standard is a model and a reviewer, and prose is the right format for both; grinding a rule into assertions loses the rationale that lets an agent apply it to a case the rule did not anticipate, and the checks that can be deterministic are better written as code than derived from a paragraph. What punt-kit is, is the manager for the org's kit — the standards, the tools, the workflows, the compliance checks. The part I am investing in is `/punt:auto`. Every recurring prompt I type by hand is a program I have not written yet. Turning those into playbooks, and then into state machines with guards and resume, means the tool writes the prompt and the agent runs the loop. That is the leverage. Whether anyone outside Punt Labs wants this is a question for later, deliberately."
>
> — **J. Freeman**, Creator, Punt Labs

Call to Action

punt-kit is public at `github.com/punt-labs/punt-kit`, on PyPI as `punt-kit`, and on the punt-labs Claude Code plugin marketplace as `punt@punt-labs`, under the MIT license. It is public because the org's other tools install it, not because it is being marketed. For a Punt Labs repo that has not adopted it: run `punt audit`, fix what it reports, and put the standards reference in `CLAUDE.md`.

Frequently Asked Questions

External FAQs

These are the questions the product's actual users ask — the operator and the org's agents. "External" here means outward from the tool, not outward from Punt Labs.

Q1: What is punt-kit and who is it for?

punt-kit is the kit manager for the Punt Labs org. It manages four things: **standards** (author them, activate them in a repo, audit against them), **tools** (install them, enable them, compose their guidance into a session), **workflows** (the release engine and the playbooks), and **compliance** (the conformance audit and the PII scan). Its users are one operator and the AI agents that work roughly thirty repositories. Two surfaces are shipped — the `punt` CLI and the `punt@punt-labs` Claude Code plugin — and two more are committed but not yet built: an MCP server and a REST surface (see FAQ 7).

Q2: What does "kit manager" mean?

A repo declares which of the org's tools it uses, and punt-kit manages that declaration. Per `standards/tool-enable-disable.md`: each tool owns a `.punt-labs/<tool>/` subtree in the repo, an `enabled` marker records the choice, and the host `CLAUDE.md` carries exactly one `@`-import line per enabled tool — so the tool's own user guide, not a block of managed text, is what lands in the session. `enable` and `disable` are the verbs, and `disable` is non-destructive: it removes the import and the marker and leaves the subtree dormant.

The convention is deployed, not theoretical: biff, ethos, and vox each implement the verbs in their own CLI, and committed `enabled` markers exist in nine repos for biff and three for ethos. What punt-kit does not yet have is the manager surface itself — there is no `punt install` and no `punt enable` at v0.11.2. That is the Should Do investment (Feature 8), and the press release names this pillar as a direction for exactly that reason.

The mechanism is dispatch, not ownership. When the manager surface arrives, `punt install <tool>` will run that tool's own `install.sh`, and `punt enable <tool>` will delegate to `<tool> enable`. punt never writes another tool's vendored zone, marker, or import line itself. That is not a convenience choice — `tool-enable-disable.md` §2.2 gives each tool ownership of the vendored zone and the `enabled` marker of its own subtree, and §2.3 puts the `enable/disable` verbs on every tool's own CLI. A manager that wrote those files directly would be a second writer of state the standard assigns to one owner, and would drift the moment a tool changed what it deposits. punt-kit's job is to know what the kit is and to call the right verb on each member of it.

One naming point, since the standard already claims the bare form: §2.10 defines `punt enable` with no argument as punt enabling *itself* in a repo, like any other tool. The manager form takes an argument — `punt enable <tool>` — and dispatches.

Q3: How do the standards actually reach an agent?

Three mechanisms, none of which involve parsing prose into assertions:

- **Reference.** A repo's `CLAUDE.md` links the standards it is bound by; the agent reads the document. 26 of the org's 30 `CLAUDE.md` files reference punt-kit today.
- **Auto-load.** `lang-rules/` holds per-language coding rules (Python, Go, C, Swift) that load automatically when an agent touches a matching file, scoped by `paths:` frontmatter.

- **Check.** `punt audit` is hand-written deterministic Python that verifies the conventions a machine can verify — CI workflows, tool config, permissions, Makefile targets, README SHA pins, plugin version sync, branch-protection rulesets — and reports each finding with its rule.

The corpus is prose because its consumers are a language model and a human reviewer. Grinding a standard into assertions would lose the rationale that makes an agent apply it correctly to a case the rule did not anticipate, and the deterministic subset is cheaper to write directly as code.

Q4: What is `/punt:auto` and why is it the centerpiece?

`/punt:auto` runs **playbooks**: YAML programs whose steps are either `script` (deterministic shell) or `llm` (scoped judgment), each with an optional postcondition and a failure strategy (`retry`, `diagnose`, `abort`). The executor is an LLM following `skills/auto/SKILL.md`, which is why a judgment step can sit next to a shell step in one program — a daemon could not run half of these steps. Six playbooks ship today; the release playbook is the proof, running the 11-phase engine with LLM diagnosis on failure.

A name collision to fix. `punt auto <target>`, the CLI subcommand, renders and merges managed sections in project files (Makefile, `settings.json`). `/punt:auto <playbook>`, the slash command, is the playbook executor, and it is the one the self-prompting work builds on. They share a word and do unrelated jobs. Naming the collision is not the same as resolving it: one of the two should be renamed before the state-machine work makes `auto` load-bearing vocabulary, and that is an open call for the operator.

The investment is turning that DSL into a **prompt state machine**. The drafted design adds named states, guarded transitions (shell checks plus scoped LLM-judgment guards for the clauses that are irreducibly judgment), `poll` ticks that compile to a `/loop` registration, and run-state persisted per instance so a machine survives a crash and resumes where it stopped. A linear playbook stays valid — it is the degenerate one-state machine.

What that buys, argued on the one loop we actually run: **the PR watch.** Today the operator re-authors it as a fresh cron prompt for every open PR — twice in one session on 2026-07-27, which is the event that motivated the design. As a machine it is a `watch` state that routes to `fixing` or `merging` on guards, with the five-clause merge gate as the guard on the second edge and `fixing` ordered first so findings never wait. Written as prose in a cron prompt, the current state lives only in the model's working context and evaporates on compaction. Written as a playbook, it is a reviewed, versioned artifact — **the tool generates the prompt, and the agent runs the loop**. It is also the first conversion target and carries a decision threshold (FAQ 13), so this claim gets tested rather than asserted.

The same three constructs express the backlog sweep (a non-terminating machine) and a mission round (a do-while with a review cycle), which is how the design checks that the vocabulary generalizes. Names for those playbooks — `sprint`, `backlog-cleanup`, and the rest — are illustrative only; the operator wants to debate them and nothing in the design depends on the choice.

Q5: How do I get started?

Install via `install.sh`, or `uv tool install punt-kit` if uv is already present. In a new repo, `punt init` scaffolds CI workflows, tool config, permissions, and a `CLAUDE.md`; it detects the project type and is idempotent, so re-running updates without overwriting customizations. In an existing repo, `punt audit` reports what diverges. `/punt:auto list` shows the available playbooks.

Q6: What happens to my data?

punt-kit runs locally against the checkout in front of it. It reads repository files and, when `gh` is authenticated, queries the GitHub API for repo settings. It transmits nothing to Punt Labs and has no telemetry today. That is about to change in one specific way: the operator wants **feature usage logged**, because there is currently no way to answer "which commands does anyone actually run?" Any such logging is local-first and internal-only by default (Feature 9).

Q7: What surfaces exist, and what is coming?

`standards/architecture.md` sets the destination: the engine-and-clients model, one engine behind thin client surfaces. punt-kit is partway there.

Shipped: the `punt` CLI, and the `punt@punt-labs` Claude Code plugin that wraps it as slash commands.

Committed, not yet built: an **MCP server** (Feature 14) exposing audit, status, and playbook launch, so an agent running outside Claude Code can call the same checks and start the same playbooks; and a **REST surface** (Feature 15) so CI jobs and dashboards can read findings programmatically rather than scraping CLI output.

The reason to build both is the property the architecture standard is after: the checks and the playbooks do not fork per caller. One engine answers a human at a terminal, an agent over MCP, and a CI job over HTTP, and a finding means the same thing in all three. They sit in Should Do behind the state-machine DSL (FAQ 16) because nothing shipping today waits on them — an ordering call, not a doubt about the destination.

Q8: Can a repo use its own standards or playbooks?

Playbooks yes, standards not yet. Playbooks resolve project-local first: `./playbooks/<name>.yaml` shadows the org-wide copy, so a repo overrides any playbook by name. The standards corpus is currently referenced from punt-kit rather than pointed at a per-repo directory — the engine and corpus are separable in principle but not yet parameterized in practice (Feature 13).

Q9: Should anyone outside Punt Labs use this?

Not today. punt-kit is public — MIT, on PyPI, on the marketplace — because the org's own tools install it, and public distribution is the cheapest way to do that. Whether it fits a team that is not Punt Labs is deferred by operator direction until the internal product has found its footing (see FAQ 10, FAQ 13).

Internal FAQs

Value & Customer

Q10: Who is the customer, and what about the market?

The customer is Punt Labs: one operator and the org's AI agents, working roughly thirty repositories. That is the whole addressable population the product is built for.

Market sizing is deferred, deliberately. punt-kit is an internal tool with zero external organizations using it. Sizing a market it is not competing in would be an exercise, not a plan — so the question waits until the internal product has found its footing, which is the operator's direction (see FAQ 9).

Q11: What evidence do we have that this is worth building?

Internal usage is the evidence now. It is real, and it is narrow — one org, one operator. Stated plainly:

- **The release engine is exercised on every release.** `punt release` drives all eleven phases, including cross-repo propagation into sibling repos, and it is invoked through `/punt:auto release`.
- **The standards corpus is consumed fleet-wide.** 29 normative documents, referenced from 26 of the org's 30 `CLAUDE.md` files, plus per-language rules that load automatically on matching files.
- **The enable/disable convention has adopters.** biff, ethos, and vox implement it; committed `enabled` markers exist in nine repos for biff and three for ethos — the convention survived contact with three independent implementations.
- **The pain the DSL targets is observed, not hypothesized.** The operator hand-authored the same PR watch loop as a fresh cron prompt twice in a single session on 2026-07-27. That specific event is what motivated the state-machine design.

The context problem is externally corroborated, which is why the standards corpus exists at all: single-file agent manifests "do not scale beyond modest codebases" [5]; a study of 2,303 context files found security specified in only 14.5% [6]; agent sessions without context files consume 28.64% more runtime and 16.58% more tokens [7].

What is missing: any evidence from outside Punt Labs. There are no external users, no interviews, and no usage data — not even internal usage data, because nothing is logged yet (Feature 9). This gap is accepted for now, not closed.

Q12: What are the reference points in this space?

These are tools worth reading, not competitors to beat. punt-kit is not competing for adoption right now, so the useful question is which ideas to borrow and which scope to stay out of.

Tool	What It Does Well	What We Take From It
projen	Standards-as-code for project config across N repos [8]	The strongest prior art for cross-repo config. We stayed with prose plus deterministic checks rather than a TypeScript project model
Backstage	Developer portal with golden-path scaffolding [9]	A warning: it needs a dedicated team to run. Scope we deliberately stay out of (Feature 16)
Trunk	Metalinter orchestrating 100+ tools [10]	The linting layer, which punt-kit orchestrates rather than reimplements (Feature 18)
Copier	Template scaffolding with lifecycle updates [11]	Idempotent re-templating, which `punt init` approximates

The one thing none of them does is run the org's recurring *procedures* as programs that mix deterministic steps with model judgment — which is where punt-kit is investing. That is not a differentiation claim; it may simply be a problem only an org run by agents has.

Q13: What is the next step to validate the direction?

Internal, and concrete:

1. **Build the state-machine DSL** and convert exactly one hand-typed loop — the PR watch — into a playbook. One loop, not three: if the construct does not carry the loop we run most, it does not carry the others.
2. **Turn on usage logging**, so the metrics below can be measured at all rather than recalled.
3. **Watch for six weeks** whether the operator stops typing the loop by hand.

Decision threshold: if after that window the operator is still hand-authoring watch prompts — because the playbook is harder to invoke, or its guards are wrong often enough to distrust — then the DSL is not the leverage, and the investment stops at playbooks as they exist today.

Technical

Q14: What are the major technical risks?

1. **The executor is an LLM, so machines are not fully deterministic.** A shell guard is a pure function of the world and routes identically every evaluation; a judgment guard is not, so the same PR state can route two ways on two ticks. Mitigation: `script` is the default guard type and `llm` is opt-in, confined to clauses that are irreducibly judgment ("are these findings addressed?"). The trade is explicit in the design and is a decision the operator ratifies, not a property that leaks in.
2. **Ticks are session-scoped.** A `poll` compiles to a `/loop` registration, which lives in the session that created it and dies with it, so a machine pauses when its session ends. Mitigation is a documented property rather than machinery: run-state is durable, and any `-resume`

invocation re-registers the tick. A paused machine loses no state and cannot act wrongly — it just does not advance.

3. **Idempotency is the playbook author's burden.** Crash/resume re-visits a state, so an action that already ran must be a no-op the second time. The design pushes this onto authors (guard the merge on the PR still being open, guard the branch delete on the branch still existing). Authors will get it wrong; the mitigation is that guards recompute from live state, so a wrong re-visit is usually an empty one.
4. **Claude Code API surface.** Plugins, skills, and `/loop` are the substrate the executor stands on, and they evolve. Mitigation: punt-kit ships a working plugin and tracks changes as ordinary maintenance; the CLI half (`init`, `audit`, `pii`, `release`) runs without any of it.

Q15: What dependencies exist on other teams or systems?

`punt release` depends on GitHub (via `gh`), PyPI, and on sibling repos being checked out locally for cross-repo propagation. Distribution depends on PyPI and the Claude Code plugin marketplace.

The deeper dependency is the playbook executor: it *is* Claude Code reading `skills/auto/SKILL.md`, not a program punt-kit ships. That is a deliberate design choice — half the steps in a real playbook are judgment or sub-agent calls that no daemon could run — but it means playbooks do not execute outside a Claude Code session today. The deterministic CLI has no such dependency, and the planned MCP server narrows the gap rather than closing it: another agent runtime will be able to launch a playbook (Feature 14), but the thing executing it is still a language model following the protocol, so the dependency becomes "some capable LLM host" rather than "Claude Code specifically."

Q16: What is the development timeline?

Shipped, through v0.11.x (August 2026):

- Standards corpus — 29 documents plus `lang-rules/` for Python, Go, C, and Swift
- `punt init` (scaffold, idempotent, project-type detection), `punt audit`, `punt pii`, `punt doctor`, `punt status`
- `punt release` — the 11-phase engine, plus `-resume-from` for restarting at any phase
- `/punt:auto` playbook executor and six playbooks; the Claude Code plugin on the punt-labs marketplace

Invested next, in priority order:

1. **Prompt state-machine DSL** — states, guarded transitions, poll ticks, run-state with crash/resume; convert the PR watch loop as the first machine (Feature 7).
2. **`punt enable` and `punt install`** — the kit-manager surface: activate a standard in a repo, install and enable a tool, per `tool-enable-disable.md` (Feature 8).
3. **Usage logging** — which commands and playbooks actually get run (Feature 9).
4. **Audit revival keyed to enabled policy** — `punt audit` checks a fixed set of conventions today; the revival scopes it to what a repo has actually enabled, so a repo is measured against its own declared kit (Feature 10).
5. **MCP server** — audit, status, and playbook launch exposed as MCP tools, so an agent outside Claude Code can call the same engine (Feature 14).
6. **REST surface** — findings over HTTP for CI jobs and dashboards (Feature 15).

Items 5 and 6 complete the engine-and-clients model that `standards/architecture.md` specifies (FAQ 7). They sit behind the first four because every one of those is load-bearing for work

happening now, while the second client surfaces pay off when something outside a Claude Code session needs the engine. That is an ordering call, not a doubt about the destination.
No dates. One operator, agent-executed work, and a queue that reorders on the operator's ruling — a dated plan would be fiction, and the last one was.

Business

Q17: Why spend time on an internal tool with no revenue?

punt-kit is free and open-source with no paid tiers and no revenue plan. The case for the time is internal leverage:

- **It is the substrate the other tools ship on.** biff, quarry, vox, and lux all document `punt release` as their release path. Time spent here is amortized across every repo that releases.
- **It is how conventions reach agents at all.** The corpus plus the `@`-import architecture is the delivery mechanism for everything the org has decided about how work is done.
- **The DSL investment targets the operator's own time.** Every recurring prompt replaced by a playbook is a task the operator stops re-deriving from memory.

The risk this leaves open is opportunity cost: hours on punt-kit are hours not spent on the products it serves. The DSL work is the sharpest form of that bet (see FAQ 13).

Q18: What are the key metrics for success?

Internal, and chosen so that a bad number means something is actually wrong:

1. **Repos with standards activated** — how many of the org's repos reference the corpus and pass `punt audit`. Today: 26 of 30 reference it; conformance is unmeasured.
2. **Releases run through the engine** — the share of releases that complete via `/punt:auto release` without the operator dropping to manual steps. Resume-from-phase counts as engine; a hand-run merge does not.
3. **Recurring prompts replaced by playbooks** — how many of the loops the operator used to type are now files that are actually invoked. This is the metric for the DSL investment.
4. **Playbook failure mode mix** — of playbook runs that fail, how many are recovered by `diagnose` versus escalated to the operator. Rising escalation means the executor is not carrying its half.

Caveat, stated up front: three of these four cannot be measured today. There is no usage logging, so "how many releases ran clean" is currently an act of recall. The metrics are honest only after Feature 9 lands — which is why it is on the invested list rather than a nice-to-have.

Q19: Why now? What has changed?

Two shifts, one external and one internal.
The work moved to agents. Nearly all code in this org is now written by AI agents in Claude Code sessions, and the agent-context infrastructure that makes that governable — `CLAUDE.md`, `@`-imports, plugins, skills, hooks — did not exist eighteen months ago. Standards that live only in a senior engineer's head do not reach an agent; standards in a document that the session imports do.
The bottleneck moved from writing code to steering loops. With execution cheap, the scarce resource is the operator's attention on which loop is running, what state it is in, and whether it may advance. That is what makes `/punt:auto` the centerpiece rather than an automation convenience: the loops are now the expensive part, and they are still being typed by hand.

Risk Assessment

Risk	Rating	Assessment
Value	High	**The whole bet rides on one unproven claim: that a prompt state machine beats a well-written prose prompt. Zero machines have run.** The design argues the case — the current state survives compaction, the guards are reviewable, run-state resumes after a crash — but argument is not evidence, and authoring a machine costs more than typing the prompt it replaces (see the usability row below). The operator asking for a thing is not evidence the operator will use it. A Medium rating would average the proven shipped half (release engine on every release, corpus referenced by 26 of 30 `CLAUDE.md` files) against the unproven investment, and the unproven half is what this rating exists to flag. If the guards are wrong often enough to be distrusted, the DSL buys nothing and the value is only what already ships (see FAQ 13).
Usability	Medium	The shipped CLI is easy: `punt init`, `punt audit`, `/punt:auto release`, no configuration. The new surface is not. Writing a state machine — states, ordered guards, idempotent visits — is a harder authoring task than typing the prompt it replaces, and the person who must do the authoring is the same operator whose time the feature is supposed to save. Authoring cost is paid once per loop and the benefit recurs, so the arithmetic works only for loops that genuinely recur. The mitigation is to ship one machine for the most-repeated loop and see whether it gets used, rather than shipping a library of them.
Feasibility	Medium	The deterministic half is proven: the CLI ships, the 11-phase release engine runs across sibling repos, 210 tests pass. The unproven half is the executor. It is a language model following a protocol document, so a judgment guard is not reproducible across evaluations, and correct crash/resume depends on playbook authors writing idempotent actions. Both are named in the design with mitigations (`script` guards by default; guards recompute from live state) rather than assumed away. Nothing depends on unproven third-party technology.
Viability	Low	As an internal tool, viability is not about revenue — there is none planned — but about whether the org can keep it. It costs one operator's attention, it pays for itself on every release it drives, and it has no hosting, service, or support obligations. The real exposures are opportunity cost (hours here are hours not on the products punt-kit serves) and concentration: it is load-bearing for every release in the org and has one maintainer.

Feature Appendix

Defines the scope boundary. Because most of Must Do already ships, this appendix does double duty: it records what the product is today, and it names what is deliberately not being built.

Must Do

What the product is. Items 1–6 ship today at v0.11.2; item 7 is the one investment that is essential rather than optional — without it, `/punt:auto` stays a script runner and the self-prompting story is not true.

Each entry is tagged with the pillar it serves, so the appendix can be checked against the four-pillar claim. Read the tags and the gap is immediate: TOOLS appears nowhere in Must Do. That is the honest picture — the convention ships, punt's manager surface does not (Feature 8).

- **F1. Standards corpus and activation** — [Standards] 29 normative prose documents in `standards/`, plus `lang-rules/` per-language rules (Python, Go, C, Swift) that agents auto-load on matching files. A repo activates them by referencing them from `CLAUDE.md`
- **F2. Deterministic conformance audit** — [Compliance] `punt audit` — hand-written checks of CI workflows, tool config, permissions, Makefile targets, README SHA pins, plugin version sync, and branch-protection rulesets, each finding reported with its rule
- **F3. Release engine** — [Workflows] `punt release` — the 11-phase pipeline from preflight through cross-repo propagation and a final verify, with `-resume-from` to restart at any phase. Runs on every release in the org
- **F4. Playbook executor** — [Workflows] `/punt:auto` — YAML programs mixing `script` (deterministic shell) and `llm` (scoped judgment) steps, with postconditions and per-step failure strategies. Cross-repo rollout lives here as `standards-rollout` and `permissions-rollout`, not as a dedicated command
- **F5. Compliance scans** — [Compliance] `punt pii` — emails, home directory paths, and `.local` hostnames, with `-staged` for pre-commit
- **F6. The two shipped client surfaces** — [All four] the `punt` CLI — `init`, `audit`, `pii`, `release`, `auto`, `doctor`, `status`, `version` — and the `punt@punt-labs` Claude Code plugin that wraps them as slash commands. Note that CLI `auto` and slash `/punt:auto` are different things; see the name collision in FAQ 4. Two further clients, MCP and REST, are committed and sit in Should Do
- **F7. Prompt state-machine DSL** — [Workflows] extend the playbook DSL with named states, guarded transitions (shell checks plus scoped LLM-judgment guards), `poll` ticks, and per-instance run-state with crash/resume — turning a hand-typed recurring prompt into a reviewed file

Should Do

Committed, not yet built, and not blocking anything that ships today. Two kinds of item live here. Some were built or drafted and then went unused — they are kept because the kit-manager identity gives them a reason to exist. The rest are the client surfaces the engine-and-clients model in `standards/architecture.md` calls for: MCP and REST are the destination, and they sit here rather than in Must Do because nothing shipping today waits on them.

- **F8. Kit-manager surface** — [Tools] `punt install <tool>` runs that tool's own `install.sh`; `punt enable <tool>` delegates to `<tool> enable`, and `disable` likewise. Dispatch, not ownership: punt never writes another tool's vendored zone, marker, or import line, because

`tool-enable-disable.md` § 2.2 gives those to the tool and § 2.3 puts the verbs on the tool's own CLI. The bare `punt enable` stays punt enabling itself, per § 2.10

F9. **Usage logging** — [All four] record which commands and playbooks actually run, locally and internal-only, so the metrics in FAQ 18 are measured rather than recalled

F10. **Audit revival keyed to enabled policy** — [Compliance + Tools] scope `punt audit` to what a repo has actually enabled, so each repo is measured against its own declared kit instead of one fixed list of conventions

F11. **Reconcile on the new engine** — [Standards] `/punt:reconcile` — LLM-driven reconciliation of a repo against changed standards, rebuilt as a playbook rather than a standalone prompt

F12. **Autopilot as a state machine** — [Workflows] the autonomous backlog loop reborn as a non-terminating machine with guarded transitions, replacing today's prose loop playbook

F13. **Custom standards directory** — [Standards] parameterize the corpus location so a repo can point at its own rules. Separable in principle today, not parameterized in practice

F14. **MCP server** — [All four] expose audit, status, and playbook launch as MCP tools over the same engine the CLI calls, so an agent running outside Claude Code gets identical checks and can start the same playbooks (FAQ 7)

F15. **REST surface** — [All four] HTTP access to audit findings and run status for CI jobs and internal dashboards, so a pipeline reads structured results instead of scraping CLI output

Won't Do

Naming what we will not build, so the boundary is a decision rather than a backlog nobody gets to. Each exclusion below stands on its own reason: either another tool does the job better, or punt-kit already does it a different way. Note what is *not* here — the MCP server and the REST surface belong in Should Do, because the engine-and-clients model is the destination.

F16. **GUI or web dashboard** — punt-kit ships no user interface of its own — a web front end would split focus and target a different user, and teams that want a portal should use Backstage. This is consistent with the planned REST surface (Feature 15): punt-kit serves findings to somebody else's dashboard; it does not build one

F17. **CI system replacement** — punt-kit generates and validates CI workflows. It does not replace GitHub Actions, GitLab CI, or any CI runner

F18. **Code parsing or linting** — punt-kit orchestrates linters (ruff, mypy, shellcheck) and validates their configuration. It does not parse source code. Teams that want a metalinter should use Trunk or MegaLinter

F19. **Package management** — validates distribution config and automates release pipelines; does not replace PyPI, npm, or uv

F20. **Standards parsed into checkable assertions** — no engine that reads the markdown corpus and extracts assertions from it. Prose is the right format for a model and a reviewer, and grinding a rule into assertions loses the rationale that lets an agent apply it to a case the rule did not anticipate; the deterministic subset is better written directly as code (Feature 2)

F21. **A dedicated `punt rollout` command** — the job is already done — `standards-rollout` and `permissions-rollout` are playbooks that propagate a change across the org today (Feature 4). A command would be a second way to do one thing

F22. **External adoption machinery** — no marketing site, no onboarding path for non-Punt-Labs teams, no third-party standards packs. Deferred by operator direction until the internal product has found its footing (FAQ 9)

References

[1] Anthropic. *Effective Context Engineering for AI Agents*. "Each new session begins with no memory of what came before". 2025. URL: https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents.

[2] Matteo Ciniselli et al. *On the Use of Agentic Coding Manifests: An Empirical Study of Claude Code*. 253 CLAUDE.md files from 242 repos; manifests optimized for code execution not org conventions; security in 8.7%. 2025. URL: https://arxiv.org/abs/2509.14744.

[3] Anthropic. *Manage Claude's Memory — Claude Code Docs*. CLAUDE.md loaded every session but is static with no feedback loop to detect drift. 2025. URL: https://code.claude.com/docs/en/memory.

[4] DeepSource. *Engineering Manager's Guide to Static Analysis*. Quantifies static analysis at 4 hours/week/engineer saved on code reviews. 2025. URL: https://deepsource.com/blog/engineering-manager-guide-to-static-analysis.

[5] Jonah Katz et al. *Codified Context: Infrastructure for AI Agents in a Complex Codebase*. 283 sessions on a 108,000-line codebase; single-file manifests do not scale; three-component infrastructure for persistent agent context. 2026. URL: https://arxiv.org/abs/2602.20478.

[6] Filipe Calegario et al. *Agent READMEs: An Empirical Study of Context Files for Agentic Coding*. 2,303 context files from 1,925 repos; security in only 14.5% of files; build commands dominate at 62.3%. 2025. URL: https://arxiv.org/abs/2511.12884.

[7] Lucas Larson et al. *On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents*. 10 repos, 124 PRs; AGENTS.md presence reduces runtime 28.64% and token consumption 16.58%. 2026. URL: https://arxiv.org/abs/2601.20404.

[8] projen contributors. *projen: Rapidly Build Modern Applications with Advanced Configuration Management*. Synthesized config files should never be manually edited; custom project types can update N repos; AWS-centric, TypeScript-first. 2024. URL: https://github.com/projen/projen.

[9] Spotify Engineering. *Celebrating Five Years of Backstage*. 3,000+ organizations; adoption stalls at under 10% in most non-Spotify orgs; requires dedicated full-time team. 2025. URL: https://engineering.atspotify.com/2025/4/celebrating-five-years-of-backstage.

[10] Trunk.io. *Trunk Code Quality: Automated Code Quality for Teams*. Metalinter running 100+ tools; CI-focused; does not scaffold, manage releases, or integrate with AI agents. 2024. URL: https://trunk.io/code-quality.

[11] Copier contributors. *Comparisons — Copier Documentation*. Lifecycle management: updating existing projects when templates evolve; confirms propagation problem is real. 2024. URL: https://copier.readthedocs.io/en/stable/comparisons/.